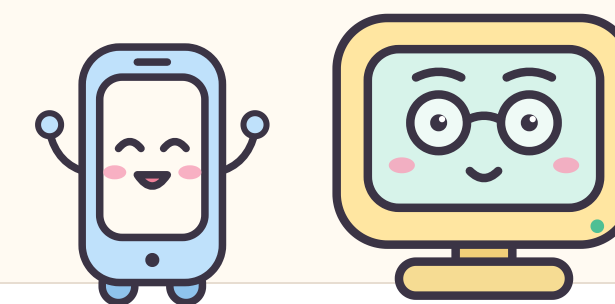


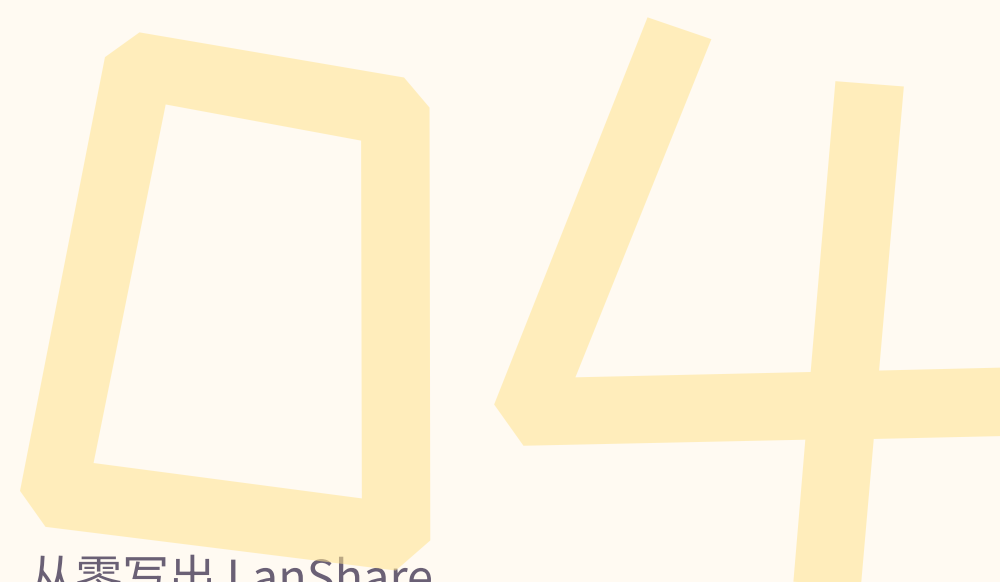


第 4 集 · 约 28 分钟

协议②：握手 与 SPAKE2

钥匙不经过网络，两边却算出了同一把

配套阅读：书的第 2 章



从零写出 LanShare

这一集你会学到

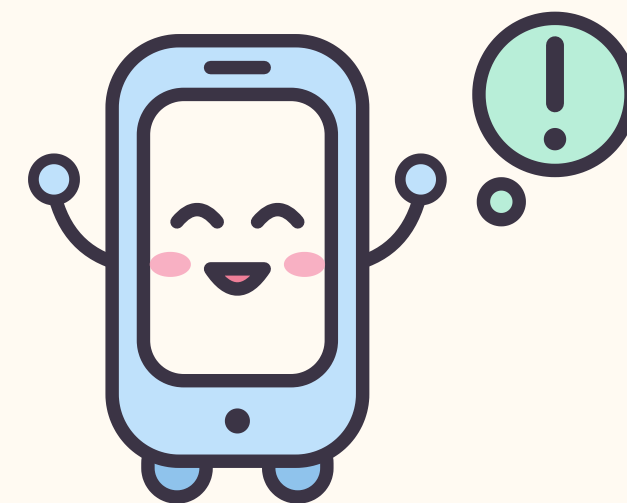
1. 二维码握手：用 HMAC 证明“我知道密钥”，又不把密钥发出去
2. HKDF：一把主密钥，派生出用途各不相同的钥匙
3. SPAKE2：只凭 6 位口令配对，抓包的人也没法离线猜
4. 已知向量：期望值要由另一份独立实现算出来



先想一个问题

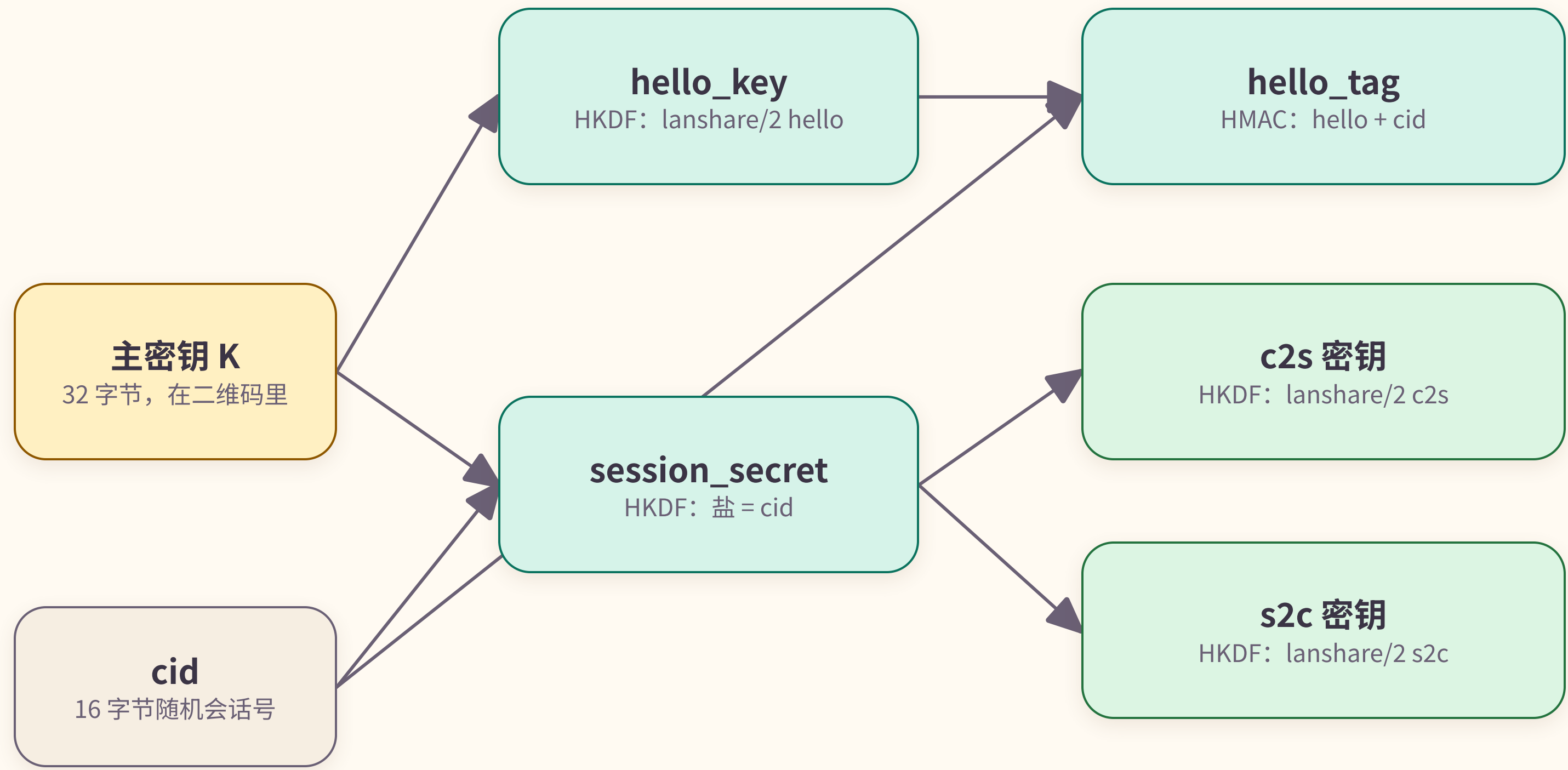
二维码里就带着密钥。小手机扫码打开网页，密钥会不会随着请求发给老电脑？

不会。密钥放在网址井号后面的片段里，按浏览器的规范，片段从来不出现在 HTTP 请求里。





一把主密钥 K，派生出所有钥匙





两个工具：HKDF 和 HMAC

● HKDF：主密钥 + 用途标签 → 一把新钥匙

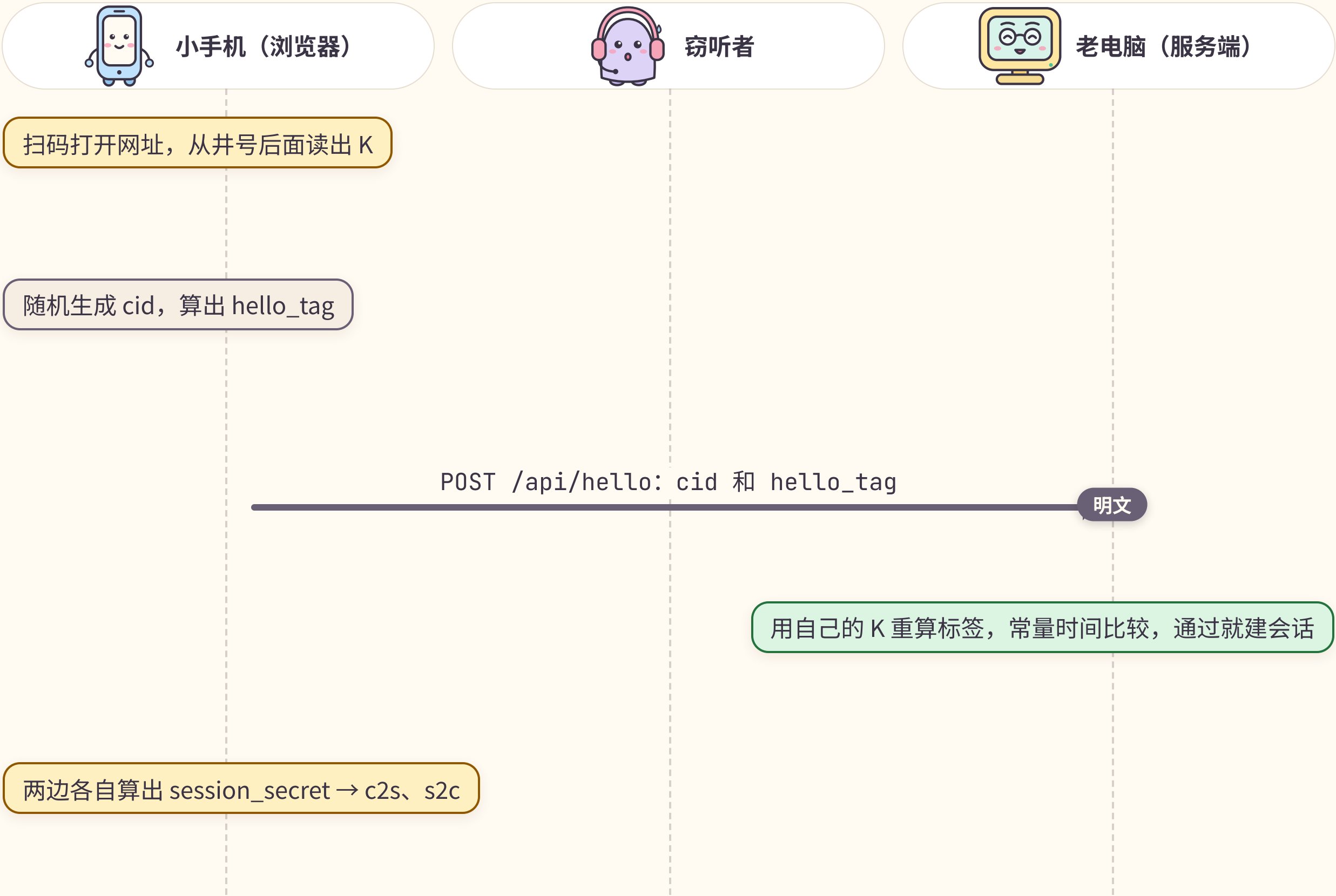
标签不同，钥匙完全不同；从新钥匙推不回主密钥

● HMAC：用密钥给消息算一个 32 字节的指纹

没有密钥算不出，也伪造不了；Python 标准库里就有



扫码握手



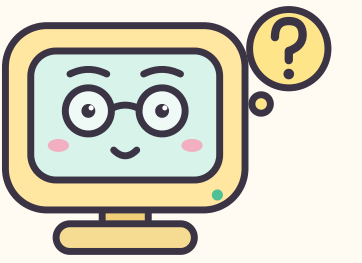
发生了什么

片段不进 HTTP 请求，K 的明文不上网

窃听器看得到 cid 和 tag，但没有 K，算不出任何密钥

会话密钥从没在网络上传过

想一想



密钥 K 在二维码里，不经过网络。那二维码泄露了，是不是就完了？



想一想

密钥 K 在二维码里，不经过网络。那二维码泄露了，是不是就完了？

对这一次运行来说，是的。二维码就是钥匙，拿到它，或者拿到口令，就成了可信用户。他能看到、下载共享文件夹里的文件，能上传，能收发文字。要是他还录下了别的设备扫码连接后的流量，也能解开，因为会话密钥由 K 和明文的 cid 派生，同一次运行里没有前向保密。但他碰不到共享文件夹以外的文件：下载只认文件夹里的文件，上传的名字会被清理；只允许本机做的操作，比如在资源管理器里显示文件，他也调用不了。补救：从托盘退出、再打开程序，K 和口令都会换新，旧二维码全部失效。所以主界面上写着“只让信任的人扫”。页面一打开就把 K 从地址栏抹掉，可扫码 App 和浏览器的历史记录里，可能还留着这个网址。这样设计的意义：秘密从“网络”这条谁都能偷听的通道，挪到了“看得见你的屏幕”这条必须在你身边的通道。WiFi 密码、蓝牙配对码，都是同一个道理。



handshake.rs: 引入和两个小工具

handshake.rs

crates/lanshare-proto/src/

第 5-24 行 · 共 24 行

```

5 use sha2::Sha256;
6
7 type HmacSha256 = Hmac<Sha256>;
8
9 /// HKDF-SHA256 派生 32 字节。`salt = None` 等价于 32 字节全 0 的盐 (RFC 5869)。
10 fn hkdf32(ikm: &[u8], salt: Option<&[u8]>, info: &str) -> [u8; 32] {
11     let mut out = [0u8; 32];
12     Hkdf::<Sha256>::new(salt, ikm)
13         .expand(info.as_bytes(), &mut out)
14         .expect("32 字节远小于 HKDF-SHA256 的输出上限");
15     out
16 }
17
18 fn mac(key: &[u8], parts: &[&[u8]]) -> HmacSha256 {
19     let mut m = HmacSha256::new_from_slice(key).expect("HMAC 接受任意长度的密钥");
20     for part in parts {
21         m.update(part);
22     }
23     m
24 }

```



握手标签与会话密钥

handshake.rs crates/lanshare-proto/src/

第 29-48 行 • 共 48 行

```

29
30 /// 二维码配对的握手标签：证明“我知道二维码里的密钥 K”，但不暴露 K。
31 pub fn hello_tag(k: &[u8; 32], cid: &[u8; 16]) -> [u8; 32] {
32     mac(&hello_key(k), &[b"hello", cid]).finalize().into_bytes().into()
33 }
34
35 /// 常量时间校验握手标签（长度不对直接不通过）。
36 pub fn verify_hello(k: &[u8; 32], cid: &[u8; 16], tag: &[u8]) -> bool {
37     mac(&hello_key(k), &[b"hello", cid]).verify_slice(tag).is_ok()
38 }
39
40 /// 二维码配对后的会话秘密：每个会话（cid）一份，由 K 派生。
41 pub fn session_secret_from_key(k: &[u8; 32], cid: &[u8; 16]) -> [u8; 32] {
42     hkdf32(k, Some(cid), "lanshare/2 session")
43 }
44
45 /// 口令（SPAKE2）配对后的会话秘密。
46 pub fn session_secret_from_pake(pake_key: &[u8], cid: &[u8; 16]) -> [u8; 32] {
47     hkdf32(pake_key, Some(cid), "lanshare/2 session")
48 }

```



校验标签：Python 和 Rust 都用常量时间

Python v1 • auth.py (校验口令)

```
1 if secrets.compare_digest(  
2     str(pin).encode("utf-8"),  
3     self.pin.encode("utf-8"),  
4 ):  
5     ... # 口令对了
```

Rust v2 • handshake.rs

```
1 mac(&hello_key(k), &[b"hello", cid])  
2     .verify_slice(tag)  
3     .is_ok()
```



两把方向密钥

handshake.rs

crates/lanshare-proto/src/

第 54-73 行 · 共 73 行

```

54     pub c2s: [u8; 32],
55     /// 服务端 → 客户端
56     pub s2c: [u8; 32],
57 }
58
59 impl ChannelKeys {
60     pub fn derive(session_secret: &[u8; 32]) -> Self {
61         Self {
62             c2s: hkdf32(session_secret, None, "lanshare/2 c2s"),
63             s2c: hkdf32(session_secret, None, "lanshare/2 s2c"),
64         }
65     }
66 }
67
68 // 故意不派生 Debug: 免得哪天被 {:?} 打进日志。
69 impl std::fmt::Debug for ChannelKeys {
70     fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
71         f.write_str("ChannelKeys(<redacted>)")
72     }
73 }

```



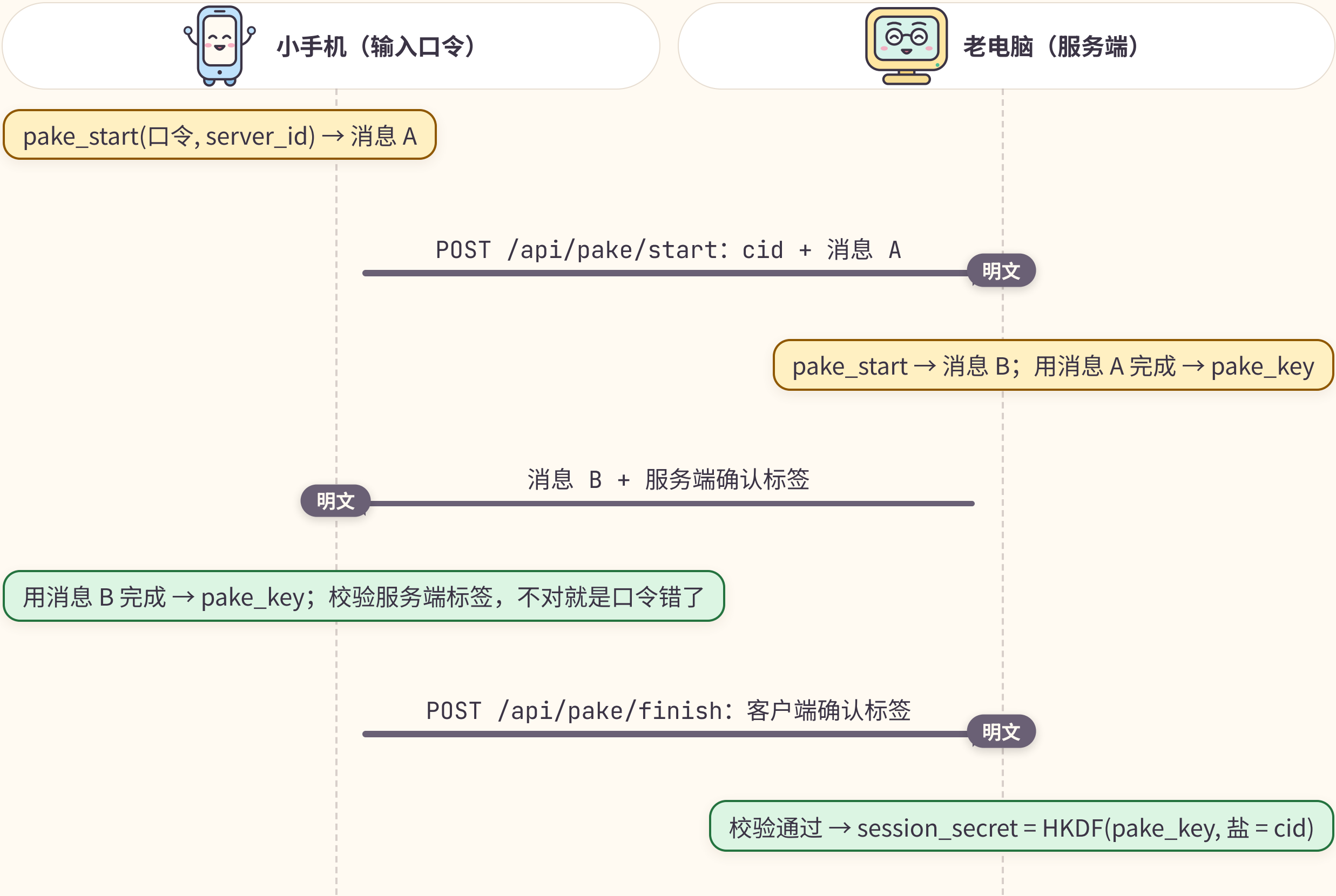
不能扫码的时候：6 位口令

- 6 位口令只有 100 万种可能
- 直接用口令派生密钥 → 抓一次包，回家离线试完 100 万种
- SPAKE2：抓到的消息没法用来验证猜测

想猜，只能真的和电脑交互一次，一次只能试一个



口令配对：SPAKE2 加确认标签



发生了什么

消息里没有能离线验证口令的信息



pake.rs: SPAKE2 的薄包装

pake.rs crates/lanshare-proto/src/

第 17-36 行 • 共 36 行

```

17 pub fn pake_start(
18     pin: &str,
19     server_id: &str,
20     rng: impl CryptoRng + RngCore,
21 ) -> (PakeState, Vec<u8>) {
22     let identity = format!("lanshare/2|{server_id}");
23     let (state, msg) = Spake2::<Ed25519Group>::start_symmetric_with_rng(
24         &Password::new(pin.as_bytes()),
25         &Identity::new(identity.as_bytes()),
26         rng,
27     );
28     (PakeState(state), msg)
29 }
30
31 /// 用对方的消息完成配对，得到 32 字节共享密钥。口令不同时也会“成功”，但两边的密钥不一样，
32 /// 所以之后还要互相出示确认标签（见 `confirm_tag`）。
33 pub fn pake_finish(state: PakeState, peer_msg: &[u8]) -> Result<[u8; 32], ProtoError> {
34     let key = state.0.finish(peer_msg).map_err(|_| ProtoError::Pake)?;
35     key.try_into().map_err(|_| ProtoError::Pake)
36 }

```



确认标签

handshake.rs

crates/lanshare-proto/src/

第 64-83 行 · 共 83 行

```

64     }
65 }
66 }
67
68 // 故意不派生 Debug: 免得哪天被 {:?} 打进日志。
69 impl std::fmt::Debug for ChannelKeys {
70     fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
71         f.write_str("ChannelKeys(<redacted>)")
72     }
73 }
74
75 /// PAKE 的确认标签: 双方证明自己算出了同一把钥匙。`role` 为 "server" 或 "client"。
76 pub fn confirm_tag(pake_key: &[u8], role: &str, cid: &[u8; 16]) -> [u8; 32] {
77     mac(pake_key, &[role.as_bytes(), cid]).finalize().into_bytes().into()
78 }
79
80 /// 常量时间校验确认标签。
81 pub fn verify_confirm(pake_key: &[u8], role: &str, cid: &[u8; 16], tag: &[u8]) -> bool {
82     mac(pake_key, &[role.as_bytes(), cid]).verify_slice(tag).is_ok()
83 }

```



lib.rs: 登记新模块和新错误

lib.rs

crates/lanshare-proto/src/

第 21–40 行 · 共 40 行

```

21
22  /// 协议层面的错误。刻意不区分“密钥错”“被篡改”“计数器错”：对攻击者一律只说解不开。
23  #[derive(Debug, Clone, Copy, PartialEq, Eq)]
24  pub enum ProtoError {
25      /// AEAD 认证失败：密钥、计数器、aad 或密文任意一处不对。
26      Decrypt,
27      /// PAKE 对端消息格式不对。
28      Pake,
29  }
30
31  impl std::fmt::Display for ProtoError {
32      fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
33          match self {
34              ProtoError::Decrypt => write!(f, "解密失败"),
35              ProtoError::Pake => write!(f, "口令配对消息无效"),
36          }
37      }
38  }
39
40  impl std::error::Error for ProtoError {}

```



测试要用的依赖

● ● ●

Cargo.toml

crates/lanshare-proto/

第 1-19 行 • 共 19 行

```
1 [package]
2 name = "lanshare-proto"
3 description = "LanShare 协议与加密核心：服务端（原生）和浏览器（WASM）共用"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [dependencies]
9 chacha20poly1305 = { version = "0.10", default-features = false, features = ["alloc"] }
10 hkdf = "0.12"
11 hmac = "0.12"
12 rand_core = "0.6"
13 sha2 = "0.10"
14 spake2 = { version = "0.4", default-features = false }
15
16 [dev-dependencies]
17 hex = "0.4"
18 rand_core = { version = "0.6", features = ["getrandom"] }
19 serde_json = "1"
```



测试：口令配对

crate: pake.rs crates/lanshare-proto/tests/

第 22-41 行 · 共 41 行

```
22 #[test]
23 fn identity_binds_the_server() {
24     let (a, msg_a) = pake_start("123456", "server-1", 0sRng);
25     let (b, msg_b) = pake_start("123456", "server-2", 0sRng);
26     assert_ne!(pake_finish(a, &msg_b).unwrap(), pake_finish(b, &msg_a).unwrap());
27 }
28
29 #[test]
30 fn messages_do_not_reveal_the_pin_and_are_fresh() {
31     let (_, m1) = pake_start("123456", "s", 0sRng);
32     let (_, m2) = pake_start("123456", "s", 0sRng);
33     assert_ne!(m1, m2, "每次随机, 抓到的消息不能复用");
34     assert!(!m1.windows(6).any(|w| w == b"123456"));
35 }
36
37 #[test]
38 fn garbage_peer_message_is_an_error() {
39     let (a, _) = pake_start("123456", "s", 0sRng);
40     assert!(pake_finish(a, b"short").is_err());
41 }
```



测试的期望值，谁来算？

- 协议是我们自己定的，期望值也用自己的代码算 → 自己验证自己
- 请一个“外人”按协议独立算一遍：Node.js 内置的 crypto，底层是 OpenSSL
- 两份独立实现逐字节一致，才算对



用 Node 和 OpenSSL 生成已知向量

● ● ●

gen-vectors.mjs

web-tests/

第 13-32 行 · 共 32 行

```

13   return Buffer.concat([c.update(plain), c.final(), c.getAuthTag()]);
14 };
15
16 const helloKey = hkdf(K, Buffer.alloc(0), "lanshare/2 hello");
17 const ss = hkdf(K, CID, "lanshare/2 session");
18 const pakeKey = Buffer.alloc(32, 0x55);
19 const out = {
20   key: hex(K), cid: hex(CID),
21   hello_key: hex(helloKey),
22   hello_tag: hex(hmac(helloKey, "hello", CID)),
23   session_from_key: hex(ss),
24   session_from_pake: hex(hkdf(pakeKey, CID, "lanshare/2 session")),
25   c2s: hex(hkdf(ss, Buffer.alloc(0), "lanshare/2 c2s")),
26   s2c: hex(hkdf(ss, Buffer.alloc(0), "lanshare/2 s2c")),
27   confirm_server: hex(hmac(pakeKey, "server", CID)),
28   confirm_client: hex(hmac(pakeKey, "client", CID)),
29   sealed_rpc: hex(seal(K, 1, Buffer.from("lanshare/2 req /api/rpc"), Buffer.from("hello LanShare"))),
30   sealed_big_ctr: hex(seal(K, 0x0102030405060708n, Buffer.from("lanshare/2 resp /api/down/x/0"), Buffer.from("局域网快传"))),
31 };
32 console.log(JSON.stringify(out, null, 2));

```



生成已知向量

```
Windows PowerShell

PS D:\code\lanshare> node web-tests/gen-vectors.mjs
{
  "key": "000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f",
  "cid": "a0a1a2a3a4a5a6a7a8a9aaabacadaeaf",
  "hello_key": "1af57d991a6d86edc1c666e3a4e0b19ceb2563187ec4b4ec559ffa865ca6d590",
  "hello_tag": "42152e475a586954cca5c87d7fe9a8e291d4989a7d6385a4ffe0f54274bd64e1",
  "session_from_key": "ef427a89250fac982656612468f8c8e14433635cc449a428692f67861796b88c",
  "session_from_pake": "f1930ffe245cef76569e1a1e3e280fe2c783479e6927216ee2b0602499fc478c",
  "c2s": "3a75bc84c4702c5049a2006d88098f3d7a6f9bc9a9e6ef4adc92ce2b7e18a04e",
  "s2c": "71e6b3978cae3638f0d878317e96aeb6770769fb239000de6d364a1e1a9c1e4e",
  "confirm_server": "3cda9f992edbd75ad789ea9bd1c36fc3ca843a26b579cac919531ea0b912ed5b",
  "confirm_client": "e1f36c60494f954fa2580eb0bdfa7c59ef31417ac93a59571d8b67fdeb662728",
  "sealed_rpc": "013910b55e2af41b4322164ec1718dca74450a162f011e6500e10507f3a8",
  "sealed_big_ctr": "0a57251d3a134f21815a577d6e50878f6bdecad5ffd493cf78cadbbc4a6f5c"
}
```





已知向量 vectors.json

```
3  "cid": "a0a1a2a3a4a5a6a7a8a9aaabacadaeaf",
4  "hello_key": "1af57d991a6d86edc1c666e3a4e0b19ceb2563187ec4b4ec559ffa865ca6d590",
5  "hello_tag": "42152e475a586954cca5c87d7fe9a8e291d4989a7d6385a4ffe0f54274bd64e1",
6  "session_from_key": "ef427a89250fac982656612468f8c8e14433635cc449a428692f67861796b88c",
7  "session_from_pake": "f1930ffe245cef76569e1a1e3e280fe2c783479e6927216ee2b0602499fc478c",
8  "c2s": "3a75bc84c4702c5049a2006d88098f3d7a6f9bc9a9e6ef4adc92ce2b7e18a04e",
9  "s2c": "71e6b3978cae3638f0d878317e96aeb6770769fb239000de6d364a1e1a9c1e4e",
10 "confirm_server": "3cda9f992edbd75ad789ea9bd1c36fc3ca843a26b579cac919531ea0b912ed5b",
11 "confirm_client": "e1f36c60494f954fa2580eb0bdfa7c59ef31417ac93a59571d8b67fdeb662728",
12 "sealed_rpc": "013910b55e2af41b4322164ec1718dca74450a162f011e6500e10507f3a8",
13 "sealed_big_ctr": "0a57251d3a134f21815a577d6e50878f6bdecad5ffd493cf78cadbbc4a6f5c"
14 }
```





已知向量测试：拷贝过来，挑几段讲

● ● ●

vectors.rs

crates/lanshare-proto/tests/ 从仓库拷贝

第 75-92 行 · 共 92 行

```
75
76 #[test]
77 fn seal_matches_openssl() {
78     let v = vectors();
79     let k = key(&v);
80     assert_eq!(seal(&k, 1, &req_aad("/api/rpc"), b"hello LanShare"), bytes(&v, "sealed_rpc"));
81     assert_eq!(
82         seal(&k, 0x0102030405060708, &resp_aad("/api/down/x/0"), "局域网快传".as_bytes()),
83         bytes(&v, "sealed_big_ctr")
84     );
85 }
86
87 #[test]
88 fn open_reverses_openssl_ciphertext() {
89     let v = vectors();
90     let plain = open(&key(&v), 1, &req_aad("/api/rpc"), &bytes(&v, "sealed_rpc")).unwrap();
91     assert_eq!(plain, b"hello LanShare");
92 }
```



跑全部测试

```
Windows PowerShell

PS D:\code\lanshare> cargo test -p lanshare-proto
    Updating crates.io index
    Locking 11 packages to latest Rust 1.96.0 compatible versions
    Adding getrandom v0.2.17
    Adding hex v0.4.3
    Adding itoa v1.0.18
    Adding memchr v2.8.3
    Adding serde v1.0.229
    Adding serde_core v1.0.229
    Adding serde_derive v1.0.229
    Adding serde_json v1.0.151
    Adding syn v3.0.6
    Adding wasi v0.11.1+wasi-snapshot-preview1
    Adding zmij v1.0.23
    Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 4.63s
..... (省略 6 行: src 里的单元测试, 0 个)
    Running tests\channel.rs (target\debug\deps\channel-ffe1e4ff30bebe6d.exe)

running 2 tests
test any mismatch fails to open ... ok
```



测试真的有用吗？故意改坏一行

```
Windows PowerShell

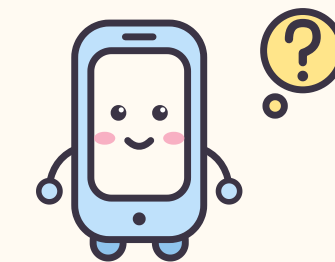
PS D:\code\lanshare> cargo test -p lanshare-proto --test vectors
   Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
   Finished `test` profile [unoptimized + debuginfo] target(s) in 1.00s
   Running tests\vectors.rs (target\debug\deps\vectors-e2b4c90670041b9f.exe)

running 8 tests
test hello_tag_matches_openssl ... ok
test channel_keys_match_openssl ... ok
test aad_layout ... ok
test verify_hello_accepts_only_the_right_tag ... ok
test session_secrets_match_openssl ... ok
test confirm_tags_match_openssl_and_differ_by_role ... ok
test seal_matches_openssl ... FAILED
test open_reverses_openssl_ciphertext ... FAILED

failures:

---- seal_matches_openssl stdout ----

thread 'seal_matches_openssl' (3136) panicked at crates\lanshare-proto\tests\vectors.rs:80:5:
assertion `left == right` failed
```



选择题

手机扫码打开 `http://IP:端口/#密钥`，这个密钥什么时候会以明文出现在网络上？

- A 打开网页时，随 GET 请求发给电脑
- B 握手时，放在 `POST /api/hello` 里发给电脑
- C 从来不会



选择题

手机扫码打开 `http://IP:端口/#密钥`，这个密钥什么时候会以明文出现在网络上？

A 打开网页时，随 GET 请求发给电脑

井号后面的片段只在浏览器本地使用，不会出现在 HTTP 请求里。

B 握手时，放在 POST `/api/hello` 里发给电脑

握手只发 cid 和 HMAC 标签，标签是证明，不是密钥。

C 从来不会

密钥在片段里，握手只发证明，会话密钥是两边各自算出来的，抓包的人永远看不到 K 的明文。登录以后，服务端还会经加密信道把 K 发给已配对的设备，方便下次直接握手，那时它是密文。



选择题

用错误的口令完成 SPAKE2，pake_finish 会怎样？

A 返回 Err，提示口令错误

B 返回 Ok，但两边的钥匙不一样

C 程序崩溃



选择题

用错误的口令完成 SPAKE2，pake_finish 会怎样？

A 返回 Err，提示口令错误

SPAKE2 本身发现不了口令不同，只要消息格式对，就会“成功”。

C 程序崩溃

正常的消息不会让它崩溃，格式不对的消息返回 Err。

B 返回 Ok，但两边的钥匙不一样

所以还需要互相出示确认标签，标签对不上，才知道口令错了。



选择题

ChannelKeys 为什么不直接 derive Debug?

- A 因为数组不能 derive Debug
- B 为了防止密钥被调试打印写进日志
- C 为了让编译更快



选择题

ChannelKeys 为什么不直接 derive Debug?

A 因为数组不能 derive Debug

32 字节数组可以 derive Debug，是我们故意不要。

C 为了让编译更快

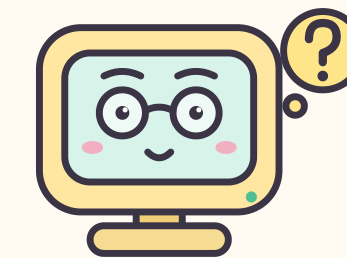
和编译速度无关，这是一个安全上的选择。

B 为了防止密钥被调试打印写进日志

手写的 Debug 只打印 ChannelKeys 和 redacted，谁打印都看不到密钥。

想一想

已知向量是 OpenSSL 算的，为什么不干脆用 Rust 算一次，把结果贴进测试，省得再写一个 JavaScript 脚本？





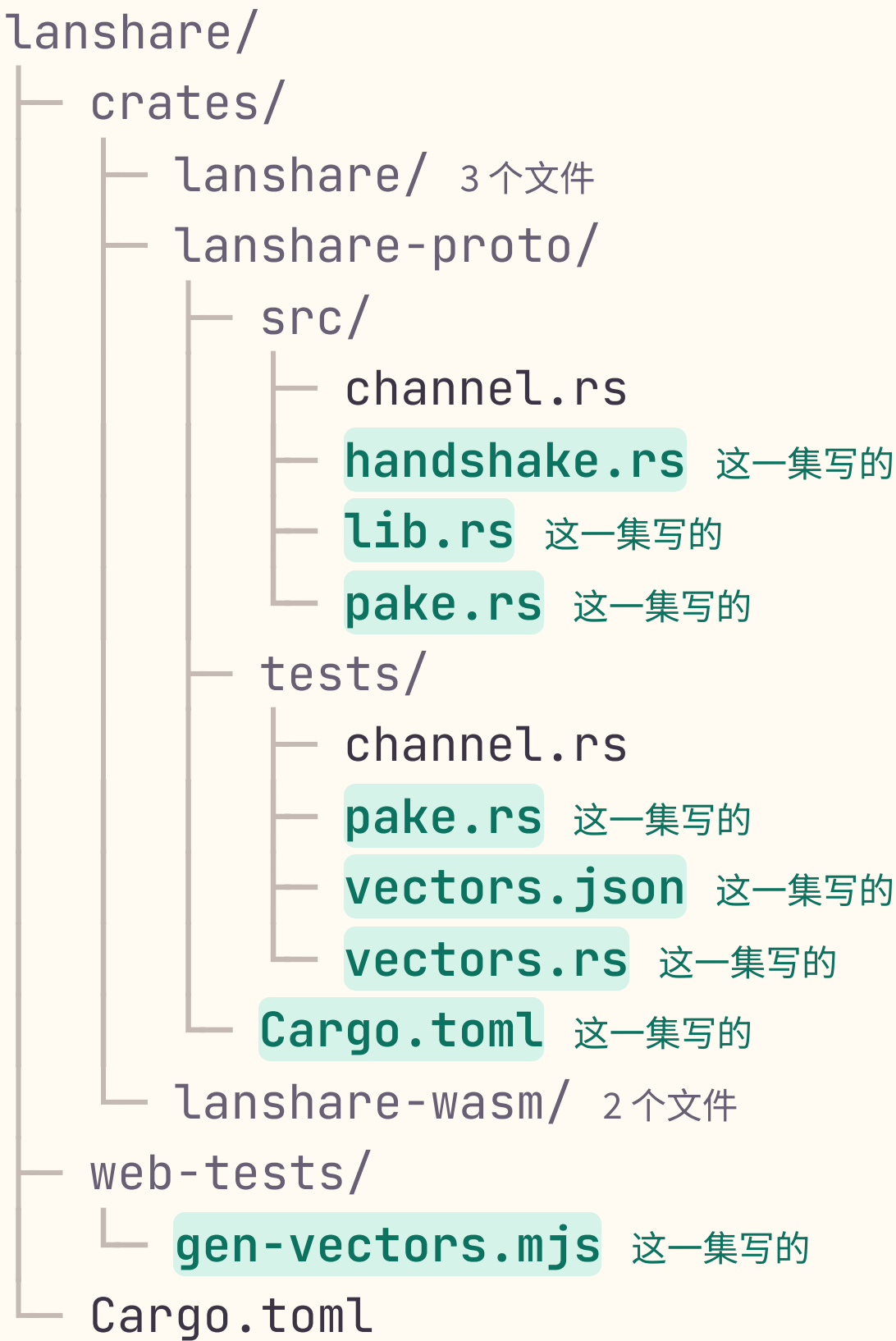
想一想

已知向量是 OpenSSL 算的，为什么不干脆用 Rust 算一次，把结果贴进测试，省得再写一个 JavaScript 脚本？

那样测试只能证明“代码和它自己一致”。比如协议标签少写了一个空格，或者字节序写反了，Rust 算出的期望值会跟着错，测试照样通过。OpenSSL 是另一份独立的实现，它和我们共同的东西只有协议文档。两边逐字节一致，说明双方对协议的理解一样，而且都没有算错。



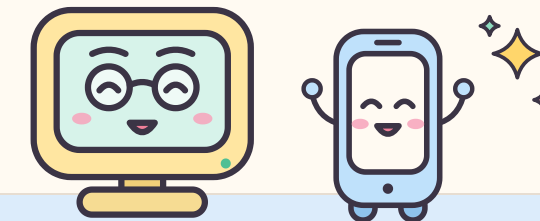
这一集写完了





这一集写了什么

- ✓ 二维码：K 放在网址片段里，明文从不上网；小手机发 HMAC 标签证明自己拥有 K
- ✓ HKDF 一把主密钥派生出所有密钥：握手、会话、两个方向，各用各的
- ✓ 校验标签用常量时间比较；装密钥的结构体手写 Debug，不进日志
- ✓ SPAKE2：错口令不报错，只是密钥不同，所以要互发确认标签
- ✓ 已知向量由 OpenSSL 独立算出，Rust 必须逐字节一致



下一集

全课完结，感谢收看